

SNP-AV1: Stacked Neural Pruners for the AV1 Intra-Frame Partition Search

Pablo Rosa

Video Technology Research Group (ViTech)
Federal University of Pelotas
Pelotas, Brazil
pablo.rosa@inf.ufpel.edu.br

Daniel Palomino

Video Technology Research Group (ViTech)
Federal University of Pelotas
Pelotas, Brazil
dpalomino@inf.ufpel.edu.br

Marcelo Porto

Video Technology Research Group (ViTech)
Federal University of Pelotas
Pelotas, Brazil
porto@inf.ufpel.edu.br

Luciano Agostini

Video Technology Research Group (ViTech)
Federal University of Pelotas
Pelotas, Brazil
agostini@inf.ufpel.edu.br

Abstract—The intra-frame coding of the AOMedia Video 1 (AV1) codec relies on a recursive block partition search that evaluates up to ten partition types at each node of the partition tree. At the highest-quality configuration this search is exhaustive by construction and dominates the intra-frame encoding time. This paper presents the Stacked Neural Pruners for AV1 (SNP-AV1), a fast decision algorithm applying two neural networks at two distinct stages of a single node of that search. The first reads thirty-six pre-search features, whose extraction cost is negligible, and decides before any rate-distortion evaluation how much of the subtree deserves to be opened. The second adds the measured rate-distortion cost of the unpartitioned node and decides whether the AB and 4-way partition types, one third of the local search time and a family no earlier predictor targeted, are evaluated at all. The two are stacked because the candidates they remove are disjoint. Experimental results show that SNP-AV1 achieves an average time savings of 18.74% with a Bjontegaard delta bitrate (BD-BR) of 0.59%, reaching 32.16% and 1.42% under a stronger calibration, and the second pruner buys time about 3.4 times cheaper than loosening the thresholds of the first. To the best of the authors’ knowledge, this is the first solution in the literature to prune the extended partitions of the AV1 intra-frame partition search with a learned model.

Index Terms—Video coding, AV1, computational effort reduction, intra-frame block partitioning, machine learning, neural networks.

I. INTRODUCTION

Video is the dominant workload of the Internet, and the share of users watching online video every month has kept growing over the last years [1]. Alongside the standards of the international bodies, such as the High Efficiency Video Coding (HEVC) [2] and the Versatile Video Coding (VVC) [3], from the Motion Picture Expert Group (ISO/IEC MPEG) and the Video Coding Experts Group (ITU-T VCEG), the Alliance for Open Media (AOMedia) maintains the AOMedia Video 1 (AV1) [4]. AV1 follows the hybrid coding flow and introduced new techniques in each of its steps [5], [6]. This coding efficiency comes at the price of computational effort: AV1 was shown to be superior to HEVC under certain conditions

[7], but encoding took twice as long [8], a problem aggravated during UHD 4K content processing.

One of the largest individual costs of AV1 intra-frame coding is the block partitioning decision. Every superblock is submitted to a recursive search that, at each node of the partition tree, evaluates up to ten partition types by rate-distortion optimization (RDO) [9] and keeps the cheapest one, descending recursively whenever the split wins. The search is exhaustive by construction, since the best partition type is only known after the node is evaluated under every candidate, and the reference software libaom [10] performs it in full at the highest-quality setting, `cpu-used=0`.

There are a few works in the literature targeting the computational effort of AV1, and none of them deals with this search. The works [11]–[13] act on the intra prediction mode decision loop, [14], [15] are hardware solutions for the AV1 intra prediction, whose target is the circuit and not the encoder search, and [16] uses decision trees on the inter-prediction modes. Learning-based partitioning solutions do exist for VVC [17]–[19] and can be adapted to the AV1 coding tools, but they were designed for the quad-tree with nested multi-type tree and evaluated on another encoder. None allows a direct and fair comparison, and none acts on the recursive partition search of AV1.

This paper presents the Stacked Neural Pruners for AV1 (SNP-AV1), a fast decision algorithm confined to the intra-frame partition search, which leaves every other encoding stage untouched. The main strategy of this solution is to remove candidates progressively, as stronger evidence becomes available: a first network decides how much of the partition tree deserves to be opened, from pre-search information whose extraction cost is negligible, and a second decides whether the most expensive family of candidates is worth evaluating, from the rate-distortion evidence the encoder has measured by then. The two are stacked because the sets of candidates they remove are disjoint, the condition under which two pruners

accumulate instead of competing for the same time. Over the eight UHD 4K video sequences of the Common Test Conditions (CTC) Class A1, SNP-AV1 reaches an average time savings of 18.74% with a Bjøntegaard delta bitrate (BD-BR) of 0.586%.

II. THE AV1 INTRA PARTITION SEARCH

The basic unit of the AV1 partitioning is the superblock, of 128×128 or 64×64 samples, and the structure that subdivides it is the partition tree [5]. AV1 supports up to ten partition types at a node [6], grouped here into four families: the two structural decisions, `PARTITION_NONE`, hereafter `NONE`, which codes the node undivided, and `PARTITION_SPLIT`, the quad-tree split into four square sub-blocks; the two rectangular types, `PARTITION_HORZ` and `PARTITION_VERT`; the four asymmetric AB types, each combining one rectangular half with two square sub-blocks; and the two 4-way types, of 4:1 aspect ratio. The AB and the 4-way families are jointly called the extended partitions.

The evaluation order inside a node is fixed and defines which information has already been paid at each point of the flow: `NONE`, the split, the rectangular types and finally the extended partitions. A predictor consulted at the pre-search point is blind to any rate-distortion information; one consulted after `NONE` reads the rate, the distortion and the RD cost of the unpartitioned node, at the price of never saving the cost of producing them.

The cost of the search was measured with the native partition timing instrumentation of libaom over 875,317 decision nodes of the 4K sequences of Section III-B, at `cpu-used=0`, taking as local work the sum of the nine non-recursive candidates of a node. It splits into three blocks of comparable size: `NONE` takes 30.1% of that time, the two rectangular types 35.6% and the six extended ones 34.3%. No single extended type exceeds 7.22%, which is why the weight of the family went unnoticed, and the cost concentrates in the large nodes, reaching 50.0% at 32×32 and 51.0% at 64×64 samples, the nodes a 4K frame produces in abundance.

This work is focused on the recursive block partitioning decision of the AV1 intra-frame coding. The other decisions taken inside a coding block, such as the intra prediction modes, the transform type search and the in-loop filtering, lie outside the partition search loop attacked here, as does the inter-frame partition search, since the solution is trained and evaluated exclusively under the All-Intra (AI) encoder configuration.

III. PROPOSED SNP-AV1 SOLUTION

The Stacked Neural Pruners for AV1 (SNP-AV1) is a fast decision algorithm that inserts two neural pruners at two distinct stages of the search of a node, as shown in Fig. 1. The main strategy of this solution is to consult each pruner at the earliest point of the flow at which the information it reads has already been paid for.

The rule behind the pairing is that two pruners in the same search compete for the same saved time whenever they remove the same candidates, however different the information

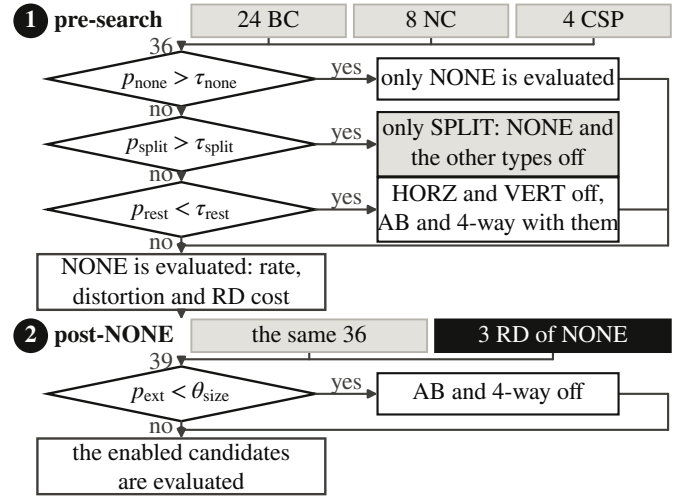


Fig. 1. Insertion points of the two pruners and the cascade of conditions each applies.

each reads, and accumulate whenever the sets they remove are disjoint. The first asks, at the pre-search point, whether the node can be settled at once on the structural axis of `NONE` against the split, so a commitment removes every competing candidate of the node and the whole subtree below it; the second asks only, and only on the nodes that survived, whether the extended partitions, the block of cost isolated in Section II, are worth evaluating. One limit follows, measured in Section IV: a node the first stage has committed never reaches the second, so the disjunction is a property of the operating point, and not of the pruners.

A. Pre-Search Feature Sets

Both stages read the same pool of pre-search features, organized into three named sets. The block-content features (BC) are twenty-four compact descriptors of the node and of its hierarchical context: variance and gradient statistics of the block and of its quadrants, edge strength and density, row and column profiles, the contrast against the parent and the sibling sub-blocks, the normalized quantization index and the position inside the 64×64 unit. The causal-neighbor partition context (NC) adds eight features from the already coded blocks above and to the left: their availability, their decided widths and heights in $\log_2$ units, and the granularity and the anisotropy of the surrounding partition structure. The coding-state and position features (CSP) add four: the effective DC dequantization step, the normalized position in the frame and the node depth. The first stage reads BC+NC+CSP, thirty-six features in all; the second reads the same thirty-six plus the logarithm of the rate, of the distortion and of the RD cost measured for `NONE`, for thirty-nine.

The criterion for inclusion was the cost of each feature where the pruner is consulted: NC and CSP are a free read of state the search already maintains, and BC is arithmetic linear in the area of the node, so no feature demands a rate-distortion evaluation of its own. Every luma sample is brought to the

eight-bit scale first, so models trained on eight-bit material are deployed unchanged on the ten-bit evaluation sequences.

B. Reference Encoding and Dataset

The reference software libaom v3.10.0 was instrumented, under a compile-time guard so that the writing binary is never the one that measures acceleration, to record at every node of the intra-frame partition search the decision taken by the encoder together with the luma samples, the causal context, the quantization state, the position and the rate-distortion result of NONE. The dataset was built from the sixteen 4K sequences of the UVG dataset [20], at 3840×2160 resolution, 8-bit depth and 4:2:0 chroma subsampling, each encoded at four quantization points (`cq-level` 20, 32, 43 and 55) over five frames sampled uniformly along the whole clip, since in AI coding consecutive frames are nearly identical. The extraction ran at `cpu-used=0`, because the label is only faithful if every partition type was evaluated: an extraction at `cpu-used=6` produced zero counts for the six extended classes, exactly the ones the second stage must predict.

The resulting dataset contains 26.98 million logged partition-tree nodes, of which 10.07 million are decision nodes of 16, 32 and 64 samples; the rest are 8×8 leaves of constant label, left out of the modeling. It was split by sequence into ten training, three validation and three reserved test sequences, frozen before any measurement, and the eight evaluation sequences belong to the CTC Class A1 set, absent from the UVG corpus, so no sequence is reused across training, testing and evaluation, which rules out data leakage.

C. Prediction Models and Training

Both stages use multilayer perceptrons (MLPs), one per node size, trained independently for the 16×16 , 32×32 and 64×64 levels; the 8×8 level is terminal and is not modeled. The topology is common to all six networks and deliberately shallow: two hidden layers with 64 and 32 ReLU units and a linear output whose softmax gives three class probabilities in the first stage and two in the second, for 27,759 parameters in total. They were trained in PyTorch [21] with the AdamW optimizer [22], a learning rate of 1×10^{-3} , a weight decay of 1×10^{-4} , batches of 4096 samples and a fixed budget of 30 epochs, under hard-label cross entropy and with no distillation term. The features are standardized during training and the standardization is folded into the first linear layer at export time, so the encoder feeds raw features and the models execute natively within the encoding pipeline; this parity was verified over 192 random vectors, with a maximum difference of 1.35×10^{-7} . Over 1,816,393 held-out decision nodes of the reserved sequences, the expected calibration error of the first stage is 0.0112, so reading its thresholds as confidence levels is verified rather than assumed.

D. Pruning Policies and Encoder Integration

The first-stage pruner applies the three actions drawn in Fig. 1 on the probabilities p_{none} , p_{split} and p_{rest} of the NONE, split and remaining classes: it terminates the node as NONE

and cuts the whole subtree when $p_{\text{none}} > \tau_{\text{none}}$, which is the primary lever of time savings; otherwise it skips NONE and recurses only into the four sub-blocks when $p_{\text{split}} > \tau_{\text{split}}$; otherwise it disables the rectangular candidates, and the extended ones with them by dependency, when $p_{\text{rest}} < \tau_{\text{rest}}$. Two operating points were frozen before the final campaign: the balanced one uses $\tau_{\text{none}} = 0.95$, $\tau_{\text{split}} = 0.90$ and $\tau_{\text{rest}} = 0.20$, the aggressive one 0.60, 0.85 and 0.40.

The second-stage pruner marks the node to skip the extended partitions when p_{ext} , the estimated probability that an extended type wins the node, falls below a threshold θ kept per node size, leaving NONE, the split and the rectangular types untouched. The threshold is per level because the values that discard the same fraction of true winners differ widely: calibrated on the reserved sequences to lose a tenth of the nodes whose optimal type is extended, the deployed values are 0.0910, 0.1031 and 0.0144 at the 16, 32 and 64-sample levels, and a stronger calibration gives 0.1776, 0.1627 and 0.0321. Both pruners were implemented in C inside libaom v3.10.0, each behind its own compile-time guard whose disabled build reproduces the reference bitstream byte for byte, which is what allows the contribution of the second to be attributed to its decisions. The policy only removes candidates and never forces an illegal one, so SNP-AV1 is fully compliant with the AV1 specification and does not affect the bitstream syntax.

IV. RESULTS AND COMPARISONS

The developed solution was evaluated using the libaom v3.10.0 AV1 reference software, following the AOM Common Test Conditions (CTC) [23] and under the AI encoder configuration. The eight 4K sequences of the CTC Class A1 test set, named in Table I, are all at 3840×2160 resolution, 10-bit depth and 4:2:0 chroma subsampling [24]. The first 15 frames of each were encoded, as the CTC specifies for the AI scenario, at `cq-level` 20, 32, 43 and 55, against the unmodified libaom at `cpu-used=0` as anchor. The BD-BR [25] is computed over the luma peak signal-to-noise ratio, and every value reported here is a coding efficiency loss. Writing $t_{\text{cfg}}(s, q)$ and $t_{\text{anc}}(s, q)$ for the encoding times of the evaluated configuration and of the anchor on sequence s at quality point q , over the sequence set S and the quality set Q ,

$$\text{TS} = \frac{1}{|S|} \sum_{s \in S} \frac{1}{|Q|} \sum_{q \in Q} \left(1 - \frac{t_{\text{cfg}}(s, q)}{t_{\text{anc}}(s, q)} \right), \quad (1)$$

reported as a percentage. All BD-BR and TS results come from this grid; the cost shares of Section II were measured on the UVG corpus instead. Every configuration was encoded once; repeating the anchor and the balanced first stage five times on Crosswalk gives a standard deviation of 0.23 percentage points on TS, so differences below twice that value are not resolved, while the BD-BR values are exact. No native speed preset is placed beside SNP-AV1: a preset reconfigures every coding stage, so a side-by-side reading would rank arrangements of different scope rather than partition-search techniques. Fig. 2 places the six measured operating points

TABLE I
PER-SEQUENCE RESULTS OF THE TWO DEPLOYED OPERATING POINTS

Sequence	Balanced		Aggressive	
	BD-BR	TS	BD-BR	TS
BoxingPractice	0.662%	20.33%	1.560%	31.97%
Crosswalk	0.534%	20.53%	1.145%	35.08%
FoodMarket2	0.609%	14.14%	1.386%	26.74%
Neon1224	0.685%	22.00%	1.427%	33.05%
NocturneDance	0.168%	13.78%	1.209%	30.79%
PierSeaSide	0.458%	17.47%	1.403%	26.03%
Tango	1.139%	22.89%	2.206%	38.80%
TimeLapse	0.433%	18.78%	0.934%	31.00%
Average	0.586%	18.74%	1.409%	31.68%

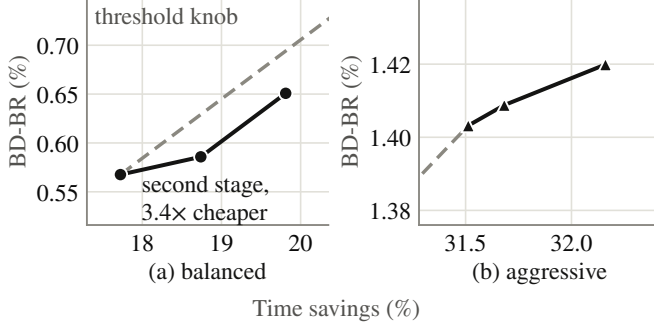


Fig. 2. Operating space of SNP-AV1 against the libaom v3.10.0 anchor at $\text{cpu-used}=0$, over the CTC Class A1 4K sequences: (a) balanced and (b) aggressive calibration, each on its own axes. Inside a panel the three points differ only in the strength of the second pruner; the dashed line is the threshold knob of the first stage.

against the anchor, and moving among them is a matter of thresholds alone, without retraining or recompiling.

The second stage is measured marginally, because it acts only on the residue of nodes the first stage did not commit. Over the balanced base it contributes 1.02 percentage points of time savings, about 4.4 times the measured standard deviation, for 0.018 percentage points of BD-BR. That is a price of 0.018 point of coding efficiency per point of time bought, against the 0.061 charged by the only alternative the deployed solution leaves to its user, which is loosening the thresholds of the first stage: the second pruner is about 3.4 times cheaper, and Fig. 2(a) shows the two prices as the two slopes that leave the same operating point. Its stronger calibration costs 0.040, still below the price of the knob, which is why the milder one is deployed. Sequence by sequence, at time savings matched by interpolating the first-stage thresholds, the stacked point wins in six of the eight sequences, two of them by strict Pareto dominance, and loses only where the extended axis is barely exercised.

This additivity is a property of the operating point, and not of the pruner. Over the aggressive base the same second stage adds 0.17 percentage points, below the resolution of the arrangement and above it in one sequence instead of six: with the NONE threshold loosened to 0.60, the first stage commits the nodes so early that they seldom reach the point at which the extended partitions would be decided, and the two pruners

fall back to disputing the same residue. That is the whole width of Fig. 2(b), whose three points cover 0.65 percentage points of time savings.

Table I opens the two deployed points sequence by sequence, and what it exposes is content dispersion, not a swept range. At the balanced calibration the time savings average $18.74\% \pm 3.40$ percentage points, from 13.78% on NocturneDance to 22.89% on Tango, and the BD-BR averages $0.586\% \pm 0.278$, from 0.168% to 1.139% on those same two sequences. The aggressive calibration keeps that shape and lifts the level, to $31.68\% \pm 4.17$ of time savings, from 26.03% to 38.80%, at $1.409\% \pm 0.377$ of BD-BR, from 0.934% to 2.206%. The dispersion is content and not measurement, since it is more than ten times the standard deviation of the repeated encodings.

Read as time bought per point of coding efficiency ceded, the eight sequences have a median of 35 points of time savings per point of BD-BR at the balanced calibration and of 22 at the aggressive one, the rising price the aggregate showed. The two columns are the two ends of one mechanism, and they correlate by rank at 0.79 ($p = 0.02$) at the balanced point: the time a sequence yields is bounded by how often the search itself settles for the unpartitioned node, and the efficiency it cedes by how often the first stage commits where that search would not have. The coupling dissolves at the aggressive point, at 0.31 ($p = 0.46$), which may be attributed to a threshold loose enough to commit nodes almost everywhere. The overhead is reported rather than omitted: feature extraction and inference together stay below one third of one percent of the encoding time, so the savings come from the pruning decisions and not from the lightness of each inference.

V. CONCLUSIONS

This paper presented SNP-AV1, a machine-learning solution that stacks two neural pruners at two distinct stages of the AV1 intra-frame partition search, reaching an average time savings of 18.74% with a BD-BR of 0.59%, and 32.16% with 1.42% under a stronger calibration. The first pruner reads thirty-six pre-search features and decides how much of the subtree is opened before any rate-distortion cost is paid; the second adds the measured RD cost of NONE and decides whether the extended partitions, one third of the local search time, are worth evaluating. Both were implemented in C inside libaom v3.10.0 and evaluated on the CTC Class A1 4K sequences under the AI configuration. Measured marginally over the balanced base, the second pruner buys time about 3.4 times cheaper than the thresholds of the first stage and wins in six of the eight sequences; the advantage collapses over the aggressive base, where the first stage commits the nodes before they reach the second, which is the operating-point limit of the stacking. Besides, to the best of the authors' knowledge, this is the first solution in the literature to prune the extended partitions of the AV1 intra-frame partition search with a learned model.

ACKNOWLEDGMENT

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001, and also by the Brazilian research support agencies CNPq and FAPERGS.

REFERENCES

- [1] Statista. (2025). *Share of Internet Users Watching Online Videos Every Month From 1st Quarter 2022 to 3rd Quarter 2024*. [Online]. Available: <https://www.statista.com/statistics/1489445/online-video-viewers-worldwide-quarterly/>
- [2] ITU-T. (2013). *H.265: High Efficiency Video Coding*. [Online]. Available: <https://www.itu.int/rec/t-rec-h.265>
- [3] ITU-T. (2020). *H.266: Versatile Video Coding*. [Online]. Available: <https://www.itu.int/rec/t-rec-h.266>
- [4] Alliance for Open Media. (2019). *AV1 Video Codec*. [Online]. Available: <https://aomedia.org/av1>
- [5] J. Han *et al.*, “A technical overview of AV1,” *Proc. IEEE*, vol. 109, no. 9, pp. 1435–1462, Sep. 2021.
- [6] Y. Chen *et al.*, “An overview of core coding tools in the AV1 video codec,” in *Proc. Picture Coding Symp. (PCS)*, Jun. 2018, pp. 41–45.
- [7] D. Grois, T. Nguyen, and D. Marpe, “Coding efficiency comparison of AV1/VP9, H.265/MPEG-HEVC, and H.264/MPEG-AVC encoders,” in *Proc. Picture Coding Symp. (PCS)*, Dec. 2016, pp. 1–5.
- [8] M. A. Layek *et al.*, “Performance analysis of H.264, H.265, VP9 and AV1 video encoders,” in *Proc. 19th Asia-Pacific Netw. Oper. Manage. Symp. (APNOMS)*, Sep. 2017, pp. 322–325.
- [9] G. J. Sullivan and T. Wiegand, “Rate-distortion optimization for video compression,” *IEEE Signal Process. Mag.*, vol. 15, no. 6, pp. 74–90, Nov. 1998.
- [10] Alliance for Open Media. *AV1 Codec Library*, v3.10.0. [Online]. Available: <https://aomedia.googlesource.com/aom>
- [11] M. Xu and B. Jeon, “Selection of intra prediction tools for fast AV1 encoding,” in *Proc. IEEE Int. Symp. Broadband Multimedia Syst. Broadcast. (BMSB)*, Oct. 2020, pp. 1–4.
- [12] M. Xu and B. Jeon, “User-priority based AV1 coding tool selection,” *IEEE Trans. Broadcast.*, vol. 67, no. 3, pp. 736–745, Sep. 2021.
- [13] J. Jeong, G. Gankhuyag, and Y.-H. Kim, “A fast intra mode decision based on accuracy of rate distortion model for AV1 intra encoding,” in *Proc. 34th Int. Tech. Conf. Circuits/Syst., Comput. Commun. (ITC-CSCC)*, Jun. 2019, pp. 1–3.
- [14] M. Corrêa, B. Zatt, D. Palomino, G. Corrêa, and L. Agostini, “A high-throughput hardware architecture for AV1 non-directional intra modes,” *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 5, pp. 1481–1494, May 2020.
- [15] L. Netto, M. Corrêa, D. Palomino, L. Agostini, and G. Corrêa, “Power-quality configurable hardware design for AV1 directional intra-frame prediction,” *IEEE Design Test*, vol. 39, no. 2, pp. 38–45, Apr. 2022.
- [16] J. Kim, S. Blasi, A. S. Dias, M. Mrak, and E. Izquierdo, “Fast inter-prediction based on decision trees for AV1 encoding,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2019, pp. 1627–1631.
- [17] M. Saldanha, G. Sanchez, C. Marcon, and L. Agostini, “Configurable fast block partitioning for VVC intra coding using light gradient boosting machine,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 6, pp. 3947–3960, Jun. 2022.
- [18] Q. He, W. Wu, L. Luo, C. Zhu, and H. Guo, “Random forest based fast CU partition for VVC intra coding,” in *Proc. IEEE Int. Symp. Broadband Multimedia Syst. Broadcast. (BMSB)*, Aug. 2021, pp. 1–4.
- [19] J. Chen, H. Sun, J. Katto, X. Zeng, and Y. Fan, “Fast QTMT partition decision algorithm in VVC intra coding based on variance and gradient,” in *Proc. IEEE Vis. Commun. Image Process. (VCIP)*, Dec. 2019, pp. 1–4.
- [20] A. Mercat, M. Viitanen, and J. Vanné, “UVG dataset: 50/120fps 4K sequences for video codec analysis and development,” in *Proc. 11th ACM Multimedia Syst. Conf. (MMSys)*, Jun. 2020, pp. 297–302.
- [21] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Dec. 2019, pp. 8024–8035.
- [22] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, May 2019, pp. 1–18.
- [23] X. Zhao *et al.*, “AOM common test conditions v9.0,” Alliance for Open Media, Codec Working Group, doc. CWG-G082, Mar. 2026.
- [24] Xiph.Org Foundation. *AOM CTC Test Set, Class A1 (4K)*. [Online]. Available: https://media.xiph.org/video/aomctc/test_set/a1_4k/
- [25] G. Bjøntegaard, “Calculation of average PSNR differences between RD-curves,” Video Coding Experts Group (VCEG), doc. VCEG-M33, Mar. 2001.